

Strata52

ABC Windows — Field Measurement Automation

Bad data physically cannot leave the job site.

The problem isn't paper. It's that incomplete or ambiguous measurements are discovered only after the technician has already left. I propose a guided, validated measurement workflow that catches problems at the source, gives the manager an exception queue instead of a re-entry job, and pushes approved, structured data straight into production, inventory, and billing.

To prove the approach rather than just describe it, I built a working prototype — technician capture app and manager dashboard, running end-to-end against a real backend.

Workflow: Before → After

Today:

```
Job assigned → Tech visits site → Measures on paper → Transcribes form
later → Manager reviews → Missing info?
    Yes → Clarification / second trip      No → Manager re-enters order
                                           ↓
                                   Production + inventory + billing
```

Information is validated too late; paper notes are unstructured and sometimes illegible; techs rely on memory transcribing later; managers manually re-key everything; missing measurements cause expensive second visits; errors propagate downstream.

Proposed:

```
Job assigned → Dynamic checklist generated → Tech captures measurements,
photos, notes → Validated live, on-device → Exceptions resolved before
leaving → Manager reviews structured submission → Approved, or
clarification requested → Sent to production, inventory, billing
```

States: Assigned → In Progress → Submitted → Needs Clarification (loops back to Submitted) → Approved → Production → Completed

If a job type has no configured measurement template, the system doesn't invent one — it routes for manual configuration rather than guessing.



[ADD DEMO VIDEO LINK HERE]

2–3 min walkthrough: tech submits a complete job → tries an incomplete one, blocked → enters an unusual measurement, flagged live → manager approves a clean one → billing queue updates → one clarification round-trip.



[ADD REPO LINK HERE]

Push to GitHub and link it here — a reviewer evaluating a build-focused hire will want to open the code, not just watch the demo.

Validation Model

Three different responses to "this might be wrong" — not all validation treated the same way:

TYPE	EXAMPLE	BEHAVIOR
Hard-required	Missing width, missing window type	Blocks submission — can never be missing
Warn + confirm	Width entered as 5"	Flagged live as the tech types; corrected, or explicitly confirmed — never a silent gap
Nudge, not block	No photo or notes	One-time, non-blocking prompt before submit — forcing uploads mid-visit isn't worth the friction

A technician can also flag any window "not confident," routing it into the manager's exception queue directly — for the cases no algorithm catches.

Tech Stack

Prototype stack: optimized for speed and demonstrating behavior. **Production stack:** selected after discovering ABC's identity, device environment, ERP, accounting, connectivity, security, and integration constraints.

What doesn't change regardless of which client this is: a shared backend is not optional — a client-only app can't make the tech→manager→billing handoff exist at all. Concretely: mobile-first PWA for capture (no app-store step, offline-capable); same app, desktop-responsive, for the manager; a backend API owning the source of truth; a real relational database (the prototype's flat JSON files are a demo shortcut, not a production plan); object storage for photos; and an API/webhook layer into ABC's existing ERP — an integration point, not a replacement.

Vendor choices (cloud, database, auth provider) should follow ABC's existing stack. Absent that information, the default is Node/Express + React + Postgres — boring, well-supported, not a locked-in recommendation.

MVP vs. MVP+

MVP: job list, guided per-window measurements, dynamic required fields, unit and plausible-range validation, optional photographs with a non-blocking nudge, manager review screen, clarification request, approval and structured downstream handoff, timestamped audit trail.

MVP+ (explicitly deferred):

- Offline-first PWA sync, and CV-assisted measurement from photos (shown *alongside* manual entry as a cross-check, not a replacement — needs a calibration reference in-frame to be accurate) — both general technical patterns, ready to design without more input from ABC.
- Voice notes converted to structured notes, fully autonomous approval, and inventory optimization — depend on decisions or systems that belong to ABC (see Open Questions), not on build time.

Configurability

PRINCIPLE	STATUS
Selecting a job loads its known info	✅ Built
Window type changes the required checklist	❌ Not built — one fixed checklist today
Different rules attached to different fields	✅ Built , but hardcoded, not configurable per job
Required vs. suspicious info behave differently	✅ Built — the validation model above

The real gap: checklist rules should vary by *both* job type and window type, not use one default set. The architecture already supports this (job metadata already flows into the form) — an extension, not a redesign.

What I Decided vs. What I Left Open

Decided: manager approval is always required, flagged or not — whether a clean submission could someday skip that step is ABC's call, not assumed here. No authentication in the prototype — a real gap, named rather than hidden.

Left open, for discovery with ABC:

- What measurement fields are required today, and do they differ by window type or job type?
- What system receives the production/inventory/billing order today — does it expose an API, or is it re-keyed by hand?
- What devices do technicians carry, and what's typical site connectivity?
- What's the current second-trip rate, and what does one actually cost?
- Who should be able to override a flagged measurement — tech, manager, or both?
- What are ABC's real authentication and security requirements for a field crew?

Success Metrics

Targets must be evaluated against baseline metrics: technician administrative time per job, manager review time per job, first-pass completeness rate, percentage of jobs requiring clarification, repeat site-visit rate, time from site visit to approval, manual data-entry time, production errors caused by incorrect measurements.

Illustrative targets, clearly labeled as hypotheses:

- Reduce technician transcription from 20 minutes to under 2 minutes
- Raise first-pass completeness above 95%
- Reduce approval latency from the following day to minutes
- Reduce measurement-related repeat visits by 70–80%

What's Real vs. Mocked

	REAL	MOCKED
Capture form, validation, exception queue, clarification loop	✓	
Approve → downstream handoff	✓ (structured payload)	Destination is a mock queue, not ABC's real ERP
Job metadata		Stands in for ABC's scheduling/CRM
Storage		Flat files here — a real deployment uses a database